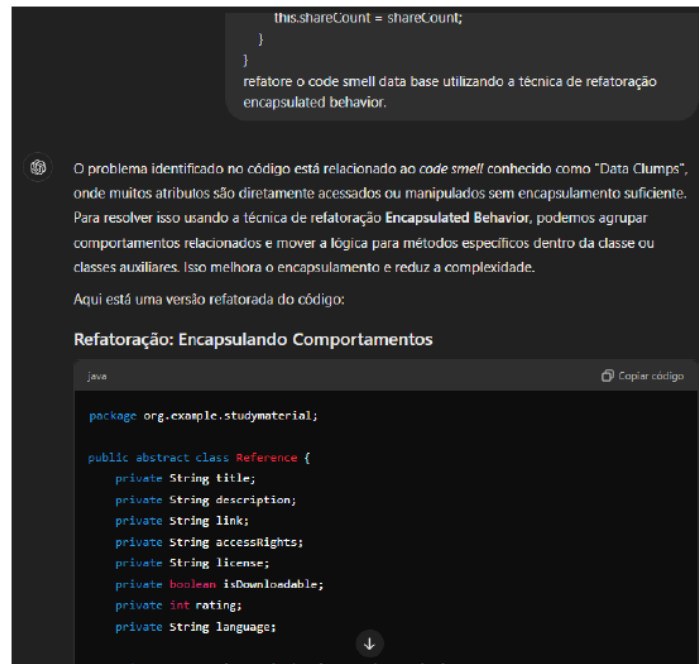


DataClass Reference

Reference

1. Foi enviado para a IA o código Reference com o seguinte comando:



```
//REFATORAÇÃO DA IA

package org.example.studymaterial;

public abstract class Reference {
    private String title;
    private String description;
    private String link;
    private String accessRights;
    private String license;
    private boolean isDownloadable;
    private int rating;
    private String language;

    private InteractionStatistics interactionStatistics;

    public Reference() {
        this.interactionStatistics = new InteractionStatistics();
    }

    // Encapsulate logic related to interaction statistics
    public void incrementViewCount() {
        interactionStatistics.incrementViewCount();
    }

    public void incrementDownloadCount() {
        interactionStatistics.incrementDownloadCount();
    }

    public void incrementShareCount() {
        interactionStatistics.incrementShareCount();
    }

    public int getViewCount() {
        return interactionStatistics.getViewCount();
    }

    public int getDownloadCount() {
        return interactionStatistics.getDownloadCount();
    }
}
```

```

    public int getShareCount() {
        return interactionStatistics.getShareCount();
    }

    // Getters and Setters for Reference attributes
    public void setTitle(String title) {
        this.title = title;
    }

    public String getTitle() {
        return title;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public String getDescription() {
        return description;
    }

    public void setLink(String link) {
        this.link = link;
    }

    public String getLink() {
        return link;
    }

    public String getAccessRights() {
        return accessRights;
    }

    public void setAccessRights(String accessRights) {
        this.accessRights = accessRights;
    }

    public String getLicense() {
        return license;
    }

    public void setLicense(String license) {
        this.license = license;
    }

    public boolean isDownloadable() {
        return isDownloadable;
    }

    public void setDownloadable(boolean downloadable) {
        isDownloadable = downloadable;
    }

    public int getRating() {
        return rating;
    }

    public void setRating(int rating) {
        this.rating = rating;
    }

    public String getLanguage() {
        return language;
    }

    public void setLanguage(String language) {
        this.language = language;
    }
}

class InteractionStatistics {
    private int viewCount;
    private int downloadCount;
    private int shareCount;

    public void incrementViewCount() {
        viewCount++;
    }
}

```

```

    }

    public void incrementDownloadCount() {
        downloadCount++;
    }

    public void incrementShareCount() {
        shareCount++;
    }

    public int getViewCount() {
        return viewCount;
    }

    public int getDownloadCount() {
        return downloadCount;
    }

    public int getShareCount() {
        return shareCount;
    }
}

```

Alterações realizadas:

1. Criação da classe `InteractionStatistics`:

- Uma nova classe foi criada para encapsular os atributos e comportamentos relacionados às estatísticas de interação: `viewCount`, `downloadCount` e `shareCount`.

2. Encapsulamento de Comportamentos:

- A lógica de incremento das estatísticas foi movida para métodos específicos dentro da classe `InteractionStatistics`. Isso melhora a coesão e centraliza as responsabilidades.

3. Redução de Acessos Diretos:

- Agora, o acesso às estatísticas de interação é feito por meio de métodos específicos da classe `Reference`, que delegam a lógica para `InteractionStatistics`.

4. Melhoria na Manutenibilidade:

- Caso seja necessário adicionar mais comportamentos ou atributos relacionados a interações, a lógica poderá ser alterada exclusivamente na classe `InteractionStatistics`, reduzindo o impacto em outras partes do código.

Essa refatoração melhora a organização do código, torna-o mais legível e reduz o acoplamento entre os atributos e comportamentos.

2. Correções de erros

```

Em:
private void editVideoAttributes(int rating, String language, int
viewCount, int shareCount, boolean isDownloadable){
    this.setRating(rating);
    this.setShareCount(shareCount);
    this.setViewCount(viewCount);
    this.setDownloadable(isDownloadable);
    this.setLanguage(language);
}

```

Seguinte erro:
 java: cannot find symbol
 symbol: method setShareCount(int)

O erro `cannot find symbol` indica que o método `setShareCount(int)` não foi encontrado na classe onde ele está sendo chamado. Isso ocorre porque, no código refatorado, o atributo `shareCount` e seus métodos (`setShareCount` e outros) foram movidos para a classe `InteractionStatistics`.

Para corrigir esse erro, você deve acessar os métodos da classe `InteractionStatistics` a partir de uma instância dessa classe, que está armazenada como o atributo `interactionStatistics` na classe `Reference`.

Aqui está a correção do método `editVideoAttributes`:

Código corrigido

```

.java Copiar código
private void editVideoAttributes(int rating, String language, int viewCount, int shareCount,
boolean isDownloadable){
    this.setRating(rating);
    this.interactionStatistics.setShareCount(shareCount);
    this.setViewCount(viewCount);
    this.setDownloadable(isDownloadable);
    this.setLanguage(language);
}

```

Envie uma mensagem para o ChatGPT

```
private void editVideoAttributes(int rating, String language, int viewCount, int shareCount, boolean isDownloadable) {
    this.setRating(rating);
    this.setLanguage(language);
    this.setDownloadable(isDownloadable);
    this.interactionStatistics.setViewCount(viewCount); // Corrigir aqui
    this.interactionStatistics.setShareCount(shareCount); // Corrigir aqui
}
```

Explicação da Correção

1. Acesso à Classe Auxiliar:

- Os métodos `setViewCount` e `setShareCount` foram movidos para a classe `InteractionStatistics`.
- Para chamá-los, você precisa acessar o atributo `interactionStatistics` da classe `Reference`.

2. Acesso aos Métodos Encapsulados:

- O código agora utiliza `this.interactionStatistics.setViewCount(viewCount)` e `this.interactionStatistics.setShareCount(shareCount)` para modificar os valores de `viewCount` e `shareCount`.

Essa alteração deve resolver o erro de compilação, já que agora o código acessa corretamente os métodos de manipulação das estatísticas de interação.

java: interactionStatistics has private access in org.example.studymaterial.Reference

O erro ocorre porque o atributo `interactionStatistics` foi declarado como `private` na classe `Reference`. Para resolver isso, você precisa acessar o atributo `interactionStatistics` de forma controlada, utilizando métodos públicos (getters e setters) na classe `Reference`.

Aqui está a solução:

Passo 1: Criar métodos de acesso para `interactionStatistics`

Na classe `Reference`, adicione métodos públicos para acessar e modificar o objeto `interactionStatistics`.

```
java Copiar código

public InteractionStatistics getInteractionStatistics() {
    return interactionStatistics;
}
```

Passo 2: Usar o getter no método `editVideoAttributes`

Atualize o método `editVideoAttributes` para usar o getter `getInteractionStatistics()`.

```
java Copiar código

private void editVideoAttributes(int rating, String language, int viewCount, int shareCount) {
    this.setRating(rating);
    this.setLanguage(language);
    this.setDownloadable(isDownloadable);
    this.getInteractionStatistics().setViewCount(viewCount); // Corrigido
    this.getInteractionStatistics().setShareCount(shareCount); // Corrigido
}
```

Passo 3: Ajustar a classe `InteractionStatistics` (se necessário)

Certifique-se de que a classe `InteractionStatistics` tenha os métodos `setViewCount` e `setShareCount` públicos.

Passo 3: Ajustar a classe `InteractionStatistics` (se necessário)

Certifique-se de que a classe `InteractionStatistics` tenha os métodos `setViewCount` e `setShareCount` públicos.

```
java Copiar código

class InteractionStatistics {
    private int viewCount;
    private int downloadCount;
    private int shareCount;

    public void setViewCount(int viewCount) {
        this.viewCount = viewCount;
    }

    public void setShareCount(int shareCount) {
        this.shareCount = shareCount;
    }

    public int getViewCount() {
        return viewCount;
    }

    public int getShareCount() {
        return shareCount;
    }
}
```

Benefícios dessa abordagem

1. **Encapsulamento Mantido:** O atributo `InteractionStatistics` continua `private`, mas agora pode ser acessado de forma controlada.
2. **Flexibilidade:** A lógica pode ser centralizada e gerenciada dentro da classe `InteractionStatistics`, facilitando futuras alterações.

Agora, o método `editVideoAttributes` deve compilar e funcionar corretamente.

Em video reference:
java: cannot find symbol
symbol: method `getIsDownloadable()`

🔗 O erro `cannot find symbol` para o método `getIsDownloadable()` indica que o método não existe na classe `Reference`. Isso ocorre porque, no código fornecido, o método foi nomeado como `isDownloadable()` e não `getIsDownloadable()`.

Como corrigir

1. Alterar a chamada do método:
 - Substitua `getIsDownloadable()` por `isDownloadable()` onde ele é chamado.
2. Explicação do erro:
 - Em Java, para atributos booleanos, os métodos de acesso são geralmente nomeados como `isAttributeName()` em vez de `getAttributeName()`.

Código Corrigido

Se você está tentando acessar o atributo `isDownloadable`, a correção seria:

```
java Copiar código

boolean downloadable = this.isDownloadable(); // Use isDownloadable() em vez de getIsDownl
```

Se você deseja seguir o padrão `getIsDownloadable()` e manter consistência com outros métodos `get`, você pode renomear o método `isDownloadable()` na classe `Reference` para `getIsDownloadable()`.

Como renomear o método

Atualize o método `isDownloadable()` na classe `Reference` para:

```
java Copiar código

public boolean getIsDownloadable() {
    return isDownloadable;
}
```

Agora, a chamada para `getIsDownloadable()` funcionará corretamente.

Resumo

- Para o padrão atual, use `isDownloadable()`.
- Se preferir o padrão `get`, renomeie o método na classe `Reference`.

```
public boolean handleStreamAvailability(){
    if(!isAvailable){
        return false;
    } else if(!this.getIsDownloadable()){
        return false;
    }
    return true;
}
```

O método `handleStreamAvailability()` tem um pequeno problema: o método `getIsDownloadable()` não existe, pois o nome correto do método na classe `Reference` é `isDownloadable()`. Para corrigir o código, você deve substituir `this.getIsDownloadable()` por `this.isDownloadable()`.

Código Corrigido

Aqui está o método corrigido:

```
java Copiar código

public boolean handleStreamAvailability() {
    if (!isAvailable) {
        return false;
    } else if (!this.isDownloadable()) {
        return false;
    }
    return true;
}
```

Melhorias no Código (Opcional)

O método pode ser simplificado para melhorar sua legibilidade e seguir as melhores práticas, removendo os `else if` e retornando diretamente os resultados.

```
java Copiar código

public boolean handleStreamAvailability() {
    return isAvailable && this.isDownloadable();
}
```

Explicação da Correção

1. Substituição de `getIsDownloadable` por `isDownloadable`:

- O método booleano para verificar o atributo `isDownloadable` é `isDownloadable()`, de acordo com a convenção Java para getters booleanos.

2. Simplificação da Lógica (Opcional):

- O operador lógico `&&` foi utilizado para retornar diretamente o resultado da avaliação das condições `isAvailable` e `isDownloadable()`.

Agora, o método funcionará corretamente e estará mais limpo.

```
java: cannot find symbol
symbol: method getIsDownloadable()
location: variable audioReference of type
org.example.studymaterial.AudioReference
```

NÃO POSSO MEXER NOS TESTES!!

Envie uma mensagem para o ChatGPT

Se você não pode alterar os testes e eles estão esperando um método chamado `getIsDownloadable()`, a solução é adicionar esse método na classe `Reference`, e garantir que ele funcione corretamente (junto com o atributo `isDownloadable`).

Solução: Adicionar o método `getIsDownloadable()`

Você pode criar um método `getIsDownloadable()` na classe `Reference`, que servirá como um alias para o método existente `isDownloadable()`.

Atualização na Classe `Reference`:

```
java Copiar código

public boolean getIsDownloadable() {
    return isDownloadable();
}
```

Agora, tanto o método `isDownloadable()` quanto o método `getIsDownloadable()` estarão disponíveis para uso, e os testes que exigem `getIsDownloadable()` passarão sem problemas.

Explicação

- O método `getIsDownloadable()` simplesmente chama `isDownloadable()`, que já está implementado.
- Isso mantém o atributo `isDownloadable` consistente e evita duplicação de lógica.
- Você não altera o comportamento do código principal, apenas adiciona compatibilidade para os testes.

Explicação

- O método `getIsDownloadable()` simplesmente chama `isDownloadable()`, que já está implementado.
- Isso mantém o atributo `isDownloadable` consistente e evita duplicação de lógica.
- Você não altera o comportamento do código principal, apenas adiciona compatibilidade para os testes.

Benefícios

- Compatibilidade com os testes existentes:** Você não precisa alterar os testes que exigem `getIsDownloadable()`.
- Sem impacto no código existente:** O método `isDownloadable()` permanece funcional e é reutilizado no novo método.

Com essa abordagem, os testes devem funcionar corretamente sem necessidade de alteração adicional.

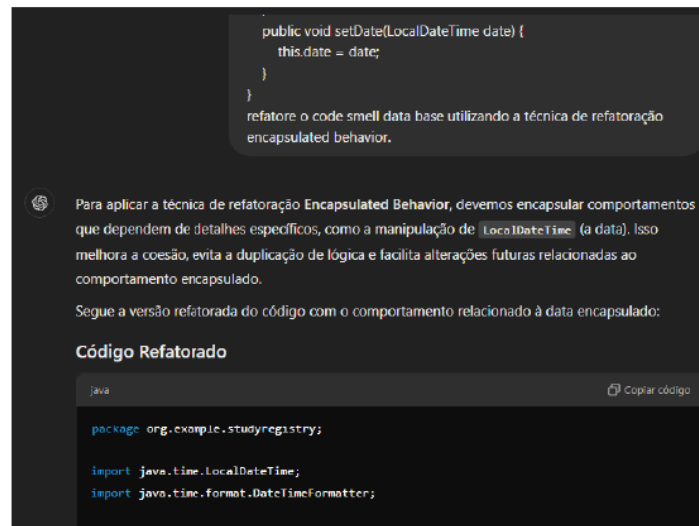
🗨️ 📄 🌟 🔄 🏠

3. Código funcional e sem code smells

DataClass Task

Task

1. Foi enviado para a IA o código Task com o seguinte comando:



```
//REFATORAÇÃO DA IA

package org.example.studyregistry;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

public class Task extends Registry {
    private String title;
    private String description;
    private String author;
    private LocalDate date;

    public Task(String title, String description, String author, LocalDate date) {
        this.title = title;
        this.name = title;
        this.description = description;
        this.author = author;
        this.date = date;
    }

    // Encapsulating behavior related to date formatting and manipulation
    public String getFormattedDate() {
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
        return this.date.format(formatter);
    }

    public boolean isDueBefore(LocalDate otherDate) {
        return this.date.isBefore(otherDate);
    }

    public boolean isDueAfter(LocalDate otherDate) {
        return this.date.isAfter(otherDate);
    }

    public void postponeByDays(int days) {
        this.date = this.date.plusDays(days);
    }

    public void advanceByDays(int days) {
        this.date = this.date.minusDays(days);
    }

    // Getters and setters for fields
    public String getTitle() {
        return title;
    }
}
```

```
}

public void setTitle(String title) {
    this.title = title;
}

public String getDescription() {
    return description;
}

public void setDescription(String description) {
    this.description = description;
}

public String getAuthor() {
    return author;
}

public void setAuthor(String author) {
    this.author = author;
}

public LocalDateTime getDate() {
    return date;
}

public void setDate(LocalDateTime date) {
    this.date = date;
}
}
```

2. Código passou nos testes tranquilamente

DataClass SearchLog

SearchLog

1. Foi enviado para a IA o código SearchLog com o seguinte comando:



```
//REFATORAÇÃO DA IA

package org.example.studysearch;

import java.util.*;

public class SearchLog {
    private final List<String> searchHistory;
    private final Map<String, Integer> searchCount;
    private boolean isLocked;
    private int numUsages;
    private String logName;

    public SearchLog(String logName) {
        this.searchHistory = new ArrayList<>();
        this.searchCount = new HashMap<>();
        this.logName = logName;
        this.numUsages = 0;
        this.isLocked = false;
    }

    // Encapsulate adding a search term and updating its count
    public void logSearch(String term) {
        if (isLocked) {
            throw new IllegalStateException("Cannot log searches; the log is locked.");
        }

        searchHistory.add(term);
        searchCount.put(term, searchCount.getOrDefault(term, 0) + 1);
        numUsages++;
    }

    // Encapsulate retrieval of search history, protecting internal list
    public List<String> getSearchHistory() {
        return Collections.unmodifiableList(searchHistory);
    }

    // Retrieve the count of a specific search term
    public int getSearchCount(String term) {
        return searchCount.getOrDefault(term, 0);
    }

    // Retrieve the most searched term(s)
    public List<String> getTopSearches() {
        int maxCount = searchCount.values().stream().max(Integer::compareTo).orElse(0);
        List<String> topSearches = new ArrayList<>();
        for (Map.Entry<String, Integer> entry : searchCount.entrySet()) {
            if (entry.getValue() == maxCount) {
                topSearches.add(entry.getKey());
            }
        }
        return topSearches;
    }

    // Toggle locking/unlocking the log
}
```

```

    public void lock() {
        isLocked = true;
    }

    public void unlock() {
        isLocked = false;
    }

    // Retrieve number of usages
    public int getNumUsages() {
        return numUsages;
    }

    // Retrieve log name
    public String getLogName() {
        return logName;
    }

    // Update log name with validation
    public void setLogName(String logName) {
        if (logName == null || logName.isBlank()) {
            throw new IllegalArgumentException("Log name cannot be null or blank.");
        }
        this.logName = logName;
    }

    public void addSearchHistory(String query) {
        searchHistory.add(query);
    }

    public void setNumUsages(int numUsages) {
        this.numUsages = numUsages;
    }
}

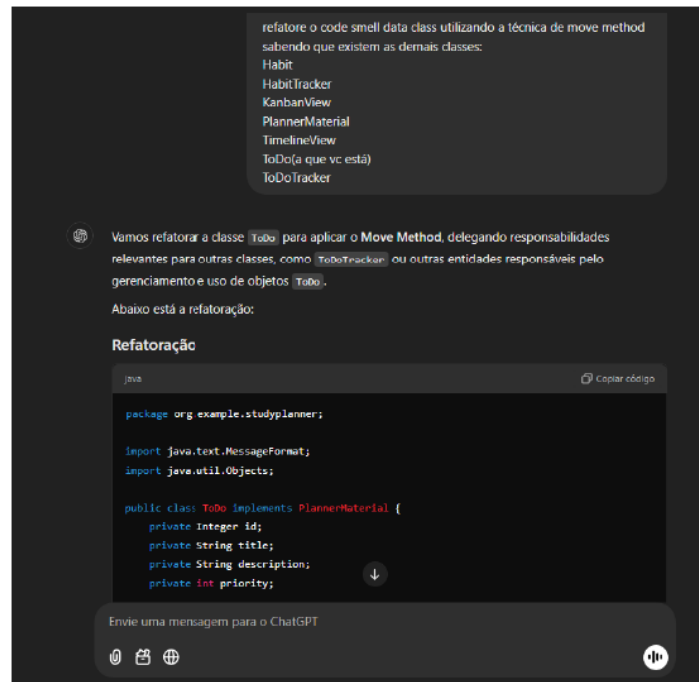
```

2. Código passou nos testes tranquilamente

DataClass ToDo

To-do

1. Foi enviado para a IA o código ToDo com o seguinte comando:



//REFATORAÇÃO DA IA

```
import java.text.MessageFormat;
import java.util.Objects;

public class ToDo implements PlannerMaterial {
    private Integer id;
    private String title;
    private String description;
    private int priority;

    public ToDo(Integer id, String title, String description, int priority) {
        this.id = id;
        this.title = title;
        this.description = description;
        this.priority = priority;
    }

    @Override
    public String toString() {
        return MessageFormat.format("[{Priority:{3}} ToDo {0}: {1}, {2}]", id, title, description, priority);
    }

    /**
     * Adds domain logic to determine if the ToDo item is high priority.
     */
    public boolean isHighPriority() {
        return priority > 7;
    }

    /**
     * Updates the description with a timestamp (encapsulates description logic).
     */
    public void updateDescription(String newDescription) {
        this.description = newDescription + " (Updated at: " + System.currentTimeMillis() + ")";
    }

    // Getters and setters
    public int getId() {
        return id;
    }
}
```

```

    }

    public void setId(Integer id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

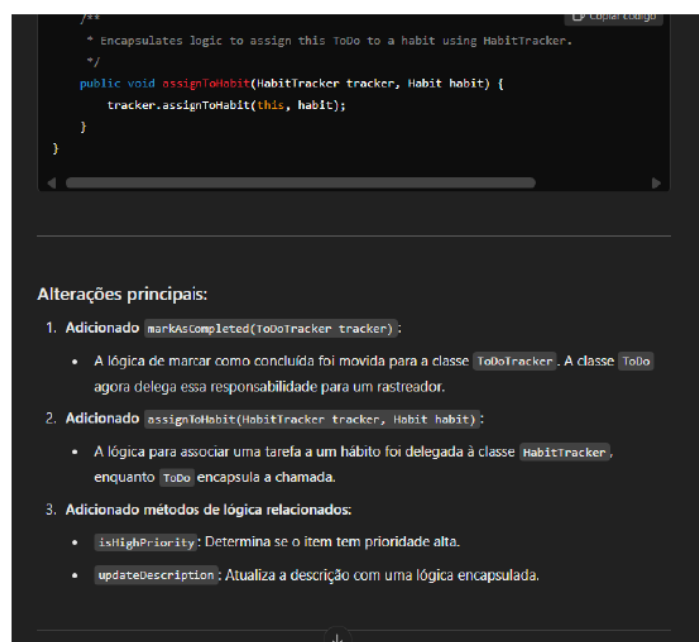
    public int getPriority() {
        return priority;
    }

    public void setPriority(int priority) {
        this.priority = priority;
    }

    /**
     * Moves responsibility for tracking ToDo items to ToDoTracker.
     */
    public void markAsCompleted(ToDoTracker tracker) {
        tracker.markCompleted(this);
    }

    /**
     * Encapsulates logic to assign this ToDo to a habit using HabitTracker.
     */
    public void assignToHabit(HabitTracker tracker, Habit habit) {
        tracker.assignToHabit(this, habit);
    }
}

```



The screenshot shows a code editor with the following Java code snippet:

```

/**
 * Encapsulates logic to assign this ToDo to a habit using HabitTracker.
 */
public void assignToHabit(HabitTracker tracker, Habit habit) {
    tracker.assignToHabit(this, habit);
}

```

Below the code, there is a section titled "Alterações principais:" (Main Changes:).

- Adicionado `markAsCompleted(ToDoTracker tracker)`:**
 - A lógica de marcar como concluída foi movida para a classe `ToDoTracker`. A classe `ToDo` agora delega essa responsabilidade para um rastreador.
- Adicionado `assignToHabit(HabitTracker tracker, Habit habit)`:**
 - A lógica para associar uma tarefa a um hábito foi delegada à classe `HabitTracker`, enquanto `ToDo` encapsula a chamada.
- Adicionado métodos de lógica relacionados:**
 - `isHighPriority`: Determina se o item tem prioridade alta.
 - `updateDescription`: Atualiza a descrição com uma lógica encapsulada.

2. Antes mesmo de conflitos a IA apresentou sugestões de ajustes

Atualizações necessárias nas outras classes:

Classe `ToDoTracker`

```
java Copiar código

package org.example.studyplanner;

import java.util.ArrayList;
import java.util.List;

public class ToDoTracker {
    private final List<ToDo> completedToDos = new ArrayList<>();

    /**
     * Marks a ToDo as completed and adds it to the tracker.
     */
    public void markCompleted(ToDo toDo) {
        if (toDo != null && !completedToDos.contains(toDo)) {
            completedToDos.add(toDo);
            System.out.println("ToDo marked as completed: " + toDo.getTitle());
        }
    }

    public List<ToDo> getCompletedToDos() {
        return completedToDos;
    }
}
```

Classe `HabitTracker`

```
java Copiar código

package org.example.studyplanner;

import java.util.HashMap;
import java.util.Map;

public class HabitTracker {
    private final Map<Habit, List<ToDo>> habitToDos = new HashMap<>();

    /**
     * Assigns a ToDo item to a habit.
     */
    public void assignToHabit(ToDo toDo, Habit habit) {
        habitToDos.computeIfAbsent(habit, k -> new ArrayList<>()).add(toDo);
        System.out.println("ToDo '" + toDo.getTitle() + "' assigned to Habit: " + habit.ge
    }

    public List<ToDo> getToDosForHabit(Habit habit) {
        return habitToDos.getOrDefault(habit, new ArrayList<>());
    }
}
```

//REFATORAÇÃO DA IA

```
package org.example.studyplanner;

import java.util.HashMap;
import java.util.Map;

public class HabitTracker {
    private final Map<Habit, List<ToDo>> habitToDos = new HashMap<>();

    /**
     * Assigns a ToDo item to a habit.
     */
    public void assignToHabit(ToDo toDo, Habit habit) {
        habitToDos.computeIfAbsent(habit, k -> new ArrayList<>()).add(toDo);
        System.out.println("ToDo '" + toDo.getTitle() + "' assigned to Habit: " + habit.getName());
    }

    public List<ToDo> getToDosForHabit(Habit habit) {
        return habitToDos.getOrDefault(habit, new ArrayList<>());
    }
}
```

Benefícios dessa refatoração:

1. Responsabilidades claras:

- `ToDo` lida apenas com seus próprios dados e lógica local.
- `ToDoTracker` gerencia o rastreamento de status e itens concluídos.
- `HabitTracker` gerencia a associação entre hábitos e tarefas.

2. Facilidade de manutenção:

- Mudanças na lógica de rastreamento ou associações podem ser feitas diretamente nos rastreadores (`ToDoTracker` e `HabitTracker`) sem modificar a classe `ToDo`.

3. Reutilização:

- As funcionalidades dos rastreadores podem ser reutilizadas por outras partes do sistema, como a `KanbanView` ou `TimelineView`.

Essa refatoração aplica o princípio de **responsabilidade única (SRP)** e distribui a lógica para as classes mais apropriadas no sistema.

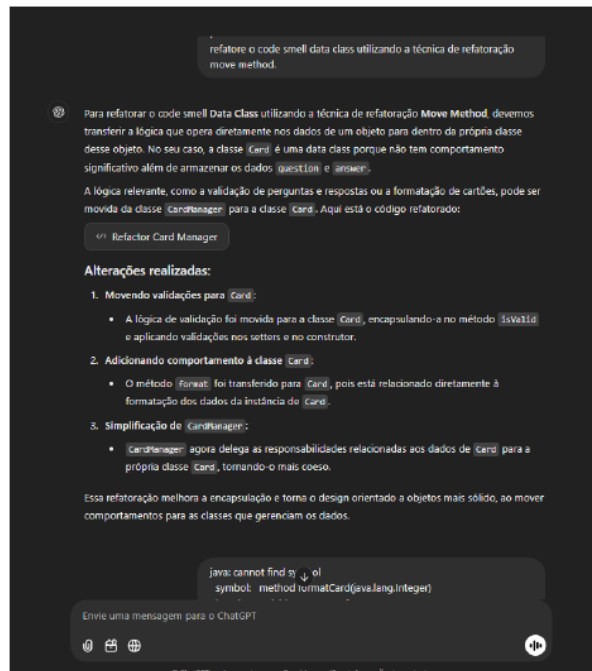


3. Nesse contextos, os ajustes sugeridos foram aplicadas de forma bruta nos códigos em questão e obteve-se sucesso na resolução do code smell

DataClass Card

CARD

1. Foi enviado para a IA o código Card e CardManager com o seguinte comando:



```
//REFATORAÇÃO DA IA
package org.example.studycards;

public class Card {
    private String question;
    private String answer;

    public Card(String question, String answer) {
        if (!isValid(question, answer)) {
            throw new IllegalArgumentException("Invalid question or answer");
        }
        this.question = question;
        this.answer = answer;
    }

    public String getQuestion() {
        return question;
    }

    public void setQuestion(String question) {
        if (!isValid(question, this.answer)) {
            throw new IllegalArgumentException("Invalid question");
        }
        this.question = question;
    }

    public String getAnswer() {
        return answer;
    }

    public void setAnswer(String answer) {
        if (!isValid(this.question, answer)) {
            throw new IllegalArgumentException("Invalid answer");
        }
        this.answer = answer;
    }

    public void edit(String question, String answer) {
        if (!isValid(question, answer)) {
            throw new IllegalArgumentException("Invalid question or answer");
        }
    }
}
```

```

    }
    this.question = question;
    this.answer = answer;
}

public String format(int id) {
    return "[id: " + id + "] Question: " + question + " Answer: " + answer;
}

private boolean isValid(String question, String answer) {
    return question != null && !question.isEmpty() && answer != null && !answer.isEmpty();
}
}

package org.example.studycards;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class CardManager {
    private Map<Integer, Card> cards;
    private Integer nextID = 1;

    private static CardManager instance = null;

    private CardManager() {
        this.cards = new HashMap<>();
    }

    public static CardManager getCardManager() {
        if (instance == null) {
            instance = new CardManager();
        }
        return instance;
    }

    public Map<Integer, Card> getCardsMap() {
        return cards;
    }

    public List<Card> getCards() {
        return new ArrayList<>(cards.values());
    }

    public List<Card> getCards(List<Integer> ids) {
        List<Card> responseCards = new ArrayList<>();
        for (Integer id : ids) {
            Card card = cards.get(id);
            if (card != null) {
                responseCards.add(card);
            }
        }
        return responseCards;
    }

    public Card getCard(Integer id) {
        return cards.get(id);
    }

    public Integer addCard(String question, String answer) {
        Card card = new Card(question, answer);
        Integer response = nextID;
        cards.put(nextID, card);
        nextID++;
        return response;
    }

    public void removeCard(Integer id) {
        cards.remove(id);
    }

    public void updateCard(Integer id, String question, String answer) {
        Card card = cards.get(id);
        if (card != null) {
            card.edit(question, answer);
        }
    }
}

```

```

    }
}

public List<String> searchInCards(String search) {
    List<String> responseCards = new ArrayList<>();
    for (int id : cards.keySet()) {
        Card card = cards.get(id);
        if (card.getQuestion().contains(search) || card.getAnswer().contains(search)) {
            responseCards.add(card.format(id));
        }
    }
    return responseCards;
}
}
}

```

2. Posteriormente tentamos resolver conflitos e erros de código pela IA enviando StudyCardsController e o erro que acusava no código

```

java: cannot find symbol
  symbol: method formatCard(java.lang.Integer)
  location: variable manager of type
  org.example.studycards.CardManager

```

O erro ocorre porque a classe `CardManager` não possui mais o método `formatCard`. Na refatoração, esse método foi removido e a responsabilidade de formatar os cartões foi transferida para a classe `Card`, com o método `format(int id)`.

Correção

Ajuste o método `handleRandomFlashCard` para usar o método `format` da instância de `Card`. Aqui está como fazer isso:

Código Corrigido:

```

java
1  public void handleRandomFlashCard() {
2      System.out.println("Random Flash card:");
3      Integer id = flashCard.randomFlashCard();
4      Card card = manager.getCard(id); // Obter o cartão pelo ID
5      if (card != null) {
6          System.out.println(card.format(id)); // Chamar o método format na instância de Card
7      } else {
8          System.out.println("card not found.");
9      }
10 }

```

Explicação

- Substituí `manager.formatCard(id)` por:
 - `manager.getCard(id)` para recuperar a instância de `Card`.
 - `card.format(id)` para formatar o cartão diretamente na classe `Card`.

Esse ajuste segue a refatoração anterior e garante que o código funcione corretamente. Certifique-se de revisar qualquer outra chamada de `formatCard` e substituí-la de forma semelhante.

3. Ainda tentamos resolver conflitos e erros de código pela IA enviando Box.Java e o erro que acusava no código:

```
public String toString() {
    CardManager manager = CardManager.getCardManager();
    StringBuilder response = new StringBuilder();
    for (Integer card : cards) {

        response.append(manager.formatCard(card)).append(System.lineSeparator());
    }
    return response.toString();
}

java: cannot find symbol
symbol: method formatCard(java.lang.Integer)
location: variable manager of type
org.example.studyCards.CardManager
```

O erro persiste porque o método `formatCard` foi removido na refatoração. Você deve substituir a chamada ao método `formatCard` pelo uso do método `format` na instância de `Card`.
Aqui está o código corrigido para o método `toString`:

Código Corrigido:

```
java Copiar código

public String toString() {
    CardManager manager = CardManager.getCardManager();
    StringBuilder response = new StringBuilder();
    for (Integer cardId : cards) { // Presumindo que 'cards' seja uma coleção de IDs de cartões
        Card card = manager.getCard(cardId); // Obter a instância do cartão
        if (card != null) {
            response.append(card.format(cardId)).append(System.lineSeparator()); // Usar o
        } else {
            response.append("[id: ").append(cardId).append("] Card not found").append(System
        }
    }
    return response.toString();
}
```

//REFATORAÇÃO DA IA

```
public String toString() {
    CardManager manager = CardManager.getCardManager();
    StringBuilder response = new StringBuilder();
    for (Integer cardId : cards) { // Presumindo que 'cards' seja uma coleção de IDs de cartões
        Card card = manager.getCard(cardId); // Obter a instância do cartão
        if (card != null) {
            response.append(card.format(cardId)).append(System.lineSeparator()); // Usar o método 'format' do Card
        } else {
            response.append("[id: ").append(cardId).append("] Card not found").append(System.lineSeparator());
        }
    }
    return response.toString();
}
```

Alterações:

- Substitua a chamada de `manager.formatCard(card)` por
 - Recuperação do cartão usando `manager.getCard(cardId)`.
 - Chamada ao método `format` no objeto `Card` retornado.
- Adicionei um tratamento para casos em que o cartão não é encontrado (`card == null`).

Certifique-se de aplicar essa lógica em qualquer outro local onde o método `formatCard` era usado anteriormente. A lógica de formatação agora está encapsulada na classe `Card` e deve ser utilizada diretamente a partir dessa classe.